A (very) Brief Introduction to Python

Politecnico di Milano

Mathematical Engineering - Financial Engineering, 2025/2026

01

What is Python?

A high-level, general-purpose language with a rich ecosystem

> *Python's simplicity, versatility, and massive ecosystem make it the go-to language for engineers, data scientists, and researchers. Its network effects mean you rarely need to build from scratch — the community has already done it.*

Python: Key Characteristics

Three properties that define the language

1 **High Level**

Easy-to-read syntax that abstracts away machine complexity — focus on problem-solving, not memory management or low-level details.

2 **Object Oriented**

Organizes code into reusable classes and objects. You can ignore this at first, but returning to it will transform how you structure larger programs.

3 **Ubiquitous**

Massive network effects: for nearly any task — data science, web, automation, finance — there is already a library that does the heavy lifting. Don't reinvent the wheel.

02

Core Building Blocks

Data types, libraries, and the Python ecosystem

Before writing programs, you need to understand how Python stores and manipulates data, and which libraries extend its capabilities into scientific computing, machine learning, and beyond.

Basic Data Types

Python's fundamental structures — indexing starts at 0

1. **Numbers** — Integers, floats, and complex numbers. e.g. 42, 3.14, 1+2j — the building blocks of all numerical computation.
2. **Strings** — Ordered collections of characters. e.g. 'Hello World' — immutable sequences with rich built-in methods for manipulation.
3. **Lists** — Ordered, mutable collections of mixed elements. e.g. [1.0, 'Hello', 0] — access with my_list[0]. The most versatile container.
4. **Dictionaries** — Key-to-value mappings. e.g. {0: 'Hello', 1: [0, 1]} — accessed via my_dict[key]. Essential for structured data storage.

Essential Python Libraries

Import with aliasing: import pandas as pd | Access sub-elements: from pandas import DataFrame

1	**datetime**	Dates and times manipulation — the standard library module for working with calendar and clock data in all Python applications.
2	**numpy**	N-dimensional arrays and scientific computing. The foundation of most numerical Python — fast, vectorized operations on large datasets.
3	**pandas**	Widely used DataFrame management for tabular data. Consider polars as a faster modern alternative.
4	**scipy**	Scientific computing: optimization, integration, and statistical functions. Built on numpy, it covers most applied mathematics needs.
5	**sklearn / others**	General-purpose machine learning. Specialized: keras/pytorch/tensorflow for deep learning; xgboost/lightgbm for gradient boosting.

03

Good Practices

Writing code that's readable, maintainable, and robust

> *Professional Python code is not just functional — it is documented, typed, and structured so that others (and your future self) can understand, maintain, and extend it with confidence.*

Documentation, Typing & Hinting

Three complementary practices that pay compounding dividends on larger codebases

1 Documentation

Write proper docstrings for functions and class public interfaces. In production code, good naming and documentation make explicit comments mostly unnecessary.

```python
class ShortInterestModel(CrossSectionalFactorModel):
    """
    Class for short interest factor models, where a long/short portfolio is built from the short
    interest factor and a set of model parameters.

    Attributes:
        long_ptf_pctiles (Tuple[float, float]): Percentiles used to build the long portfolio.
        short_ptf_pctiles (Tuple[float, float]): Percentiles used to build the short portfolio.
        weighting_scheme (WeightingScheme): Weighting scheme used to compute the long/short portfolio

    Methods:
        launch: Compute the long/short portfolio from the short interest provided.
    """
```

Documentation, Typing & Hinting

Three complementary practices that pay compounding dividends on larger codebases

2 Typing

Python is dynamically typed (operations are determined and executed at runtime), but adding type annotations (via the typing library) improves readability, signals intent, and enables static analysis tools (linters) to catch bugs before runtime.

Dynamic typing means that the code a + b returns:

- a = 2, b = 1: a + b = 3
- a = ‘a’, b = ‘b’: a + b = ‘ab’
- a = 2, b = ‘b’: a + b would raise a TypeError

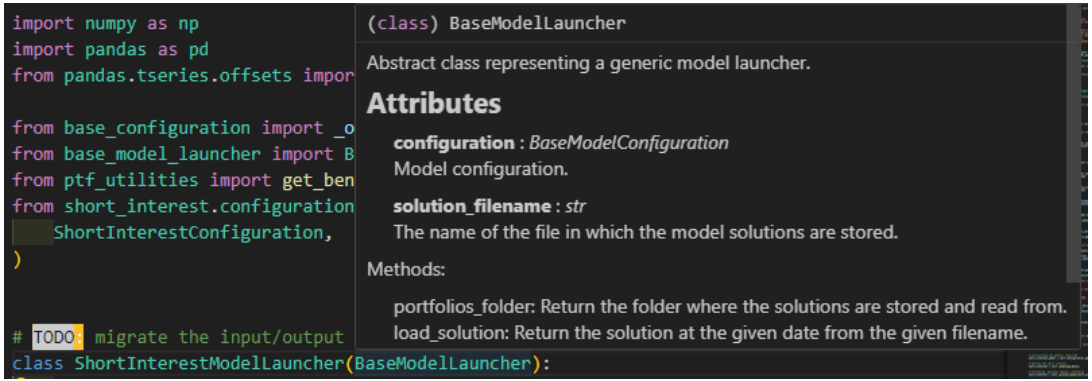
```python
def __init__(
    self,
    long_ptf_pctiles: Tuple[float, float],
    weighting_scheme: WeightingScheme,
    short_ptf_pctiles: Tuple[float, float] | None,
    inverting_pctile: float | None = None,
) -> None:
```

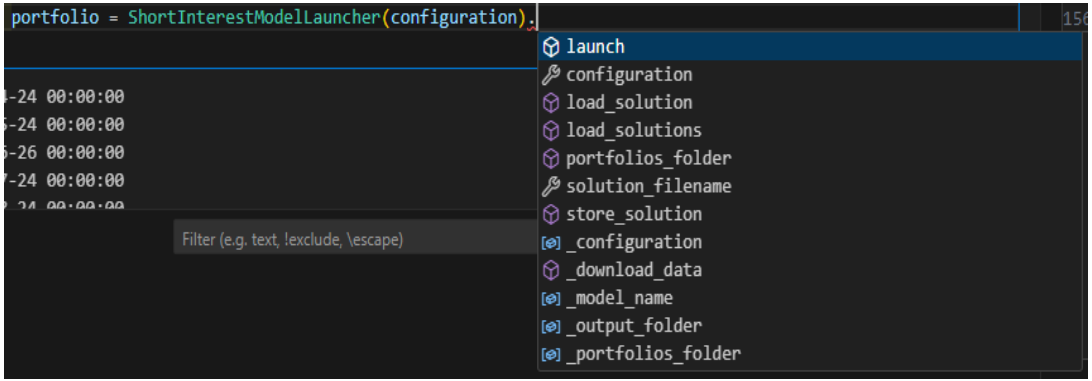
Documentation, Typing & Hinting

Three complementary practices that pay compounding dividends on larger codebases

3 IDE Hinting

Documentation + typing combine into hinting in VS Code and PyCharm: hovering shows description panels; accessing attributes shows autocomplete suggestions from inferred types.





Final Tips

1. DRY Principle: Don't repeat yourself. Write reusable functions, organize similar code into modules and packages. If you are copying code, you are doing something wrong.
2. Choose your IDE: VS Code, PyCharm, and Spyder each have trade-offs. Pick the one that fits your workflow and invest time learning its shortcuts.
3. Interactive Computing: Jupyter notebooks are natural for exploration. Use them freely for labs and analysis; switch to proper modules for production code.
4. Learn to Debug: Mastering your debugger is one of the highest-leverage skills you can develop. Learn to set breakpoints, inspect state, and step through code.
5. Version Control & Environments: Use git for all team projects. Pair it with environments (Anaconda, Poetry) to keep dependencies clean and reproducible across machines.

Resources

1. https://www.oreilly.com/content/operations-in-pandas/
2. https://ipgp.github.io/scientific_python_cheat_sheet/